

NeSyPlan Agent

A neuro-symbolic planning agent for a UR5e robot arm

Project documentation

Aleph Alpha Research GmbH

Last revised 31 August 2026

1. Overview

1.1 What NeSyPlan Agent is

A user types a sentence, for example “show me how the German flag looks by stacking the blocks in the right order of colours”, and an industrial robot arm builds it.

The language model never touches the robot. It proposes one high-level action at a time: pick the red cube, place it at grid (3, 3) on level 1. A symbolic layer running next to the robot checks that proposal against an exact model of the world. Is the cube reachable? Is that cell free? Is there anything underneath for it to rest on? Can the gripper’s jaws even open there? The layer then either performs the action or refuses it, with a written explanation, without moving the arm at all.

Everything that can be decided by computation is decided by computation. What is left for the model is what only a model can do: know what “the German flag” means, decide what to build, and work out how to recover when the world says no.

1.2 The core idea

The system rests on one claim.

“In a robotics task, most of what a language model is usually asked to do is not a language problem. It is a geometry problem, an arithmetic problem or a bookkeeping problem, and each of those has an exact algorithmic answer. Asking a language model to approximate those answers is worse than computing them, and on real hardware it is unsafe.”

So the problem is cut along the line of what is decidable. Everything on the left of the figure below is computed. Everything on the right is inferred.

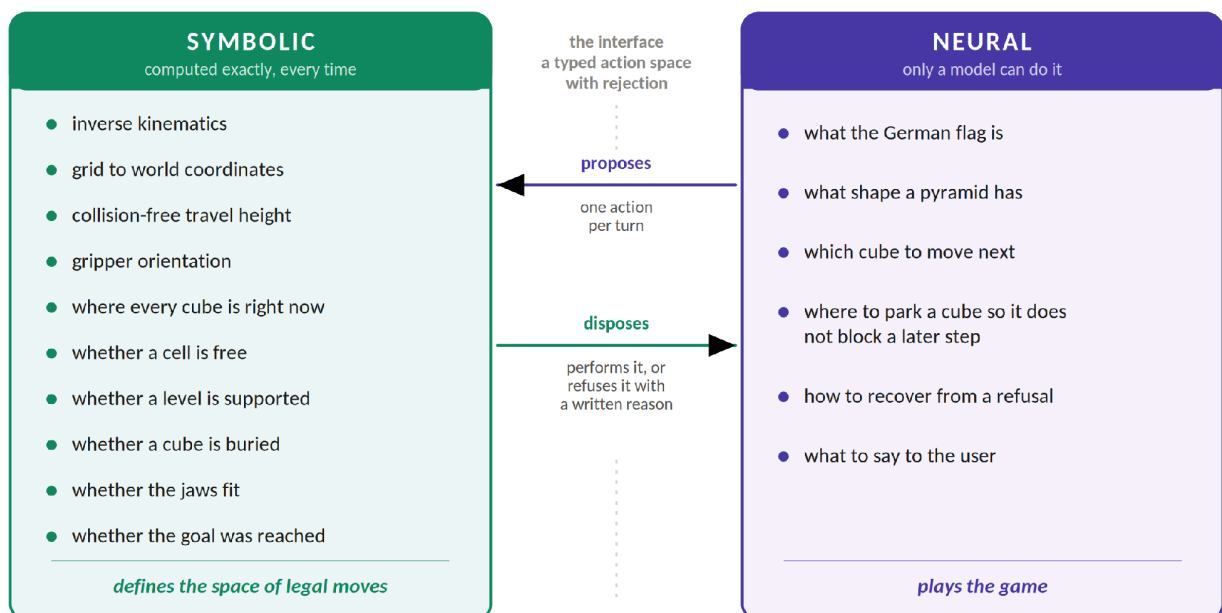


Figure 1. The dividing line. The symbolic side owns every question with an exact answer; the neural side keeps the questions that have none. The interface between them is a typed action space with rejection.

1.3 Architecture at a glance

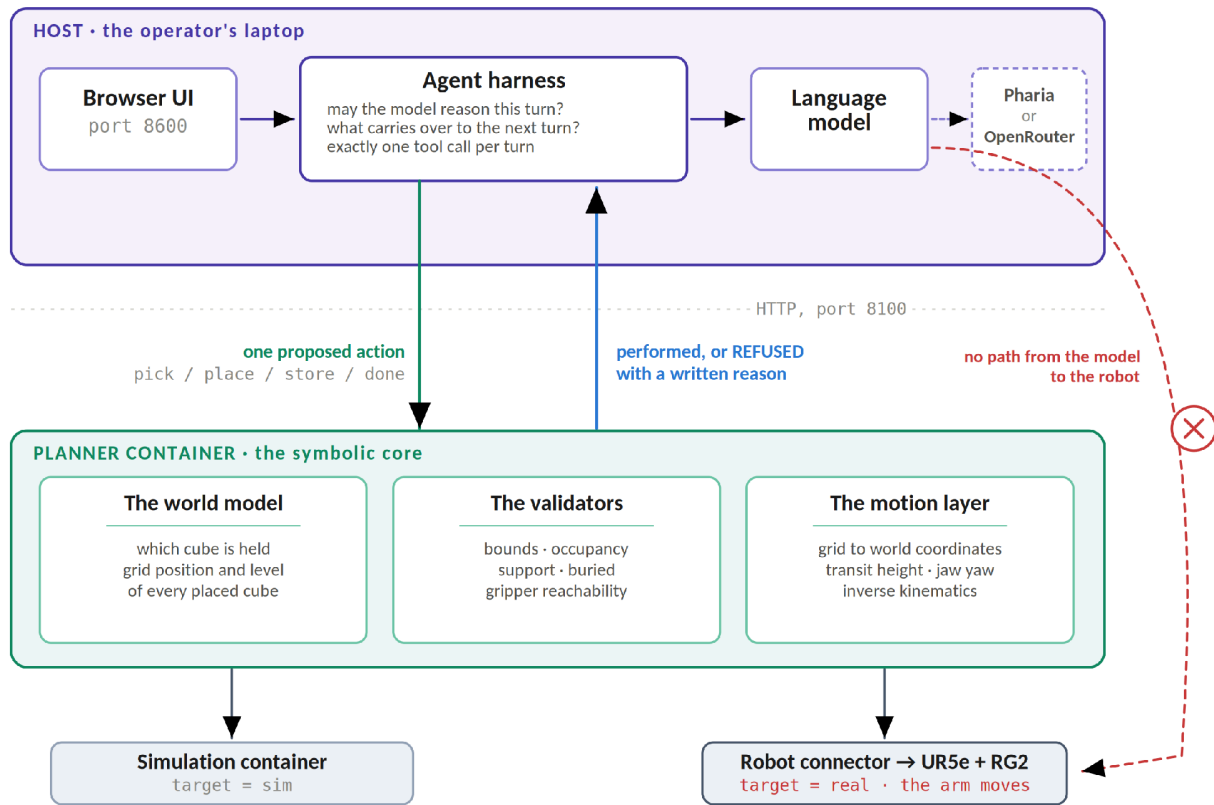


Figure 2. The system as deployed. The host runs the browser UI, the agent harness and the model. The planner container runs the symbolic core, which drives either the simulator or the physical arm.

The most important arrow in this diagram is the one that does not exist. There is no path from the language model to the robot that bypasses the symbolic core. The model's entire influence on the physical world is one small typed action object per turn, and every one of them is validated before a joint moves.

2. The hardware

The UR5e hangs from an aluminium-profile frame over the table, jaws pointing straight down. The printed A2 plate lies beneath it, carrying the 6 × 6 building area with its labelled edge and the storage row of cubes beside it, exactly the layout drawn in section 2.2. A laptop running the web demonstrator (section 6) is the entire operator interface. Nothing else is touched during a run.

The middle photograph shows why the free-axis rule of section 4.5 exists. The RG2 is a two-finger parallel gripper, so the jaws close along one horizontal axis and reach down on either side of the cube along that axis only. At the orientation shown, a cube one cell away on the open axis would be struck by a jaw before the held cube could be released. That is the case the executor detects and answers by rotating the wrist by 90 degrees.

2.1 The arm and the gripper

A Universal Robots UR5e, a six-axis collaborative arm with 5 kg payload and 850 mm reach, fitted with an OnRobot RG2 two-finger parallel gripper. On the real target the robot sits at a fixed IP on a dedicated Ethernet segment, and a connector container speaks the UR protocol to it.

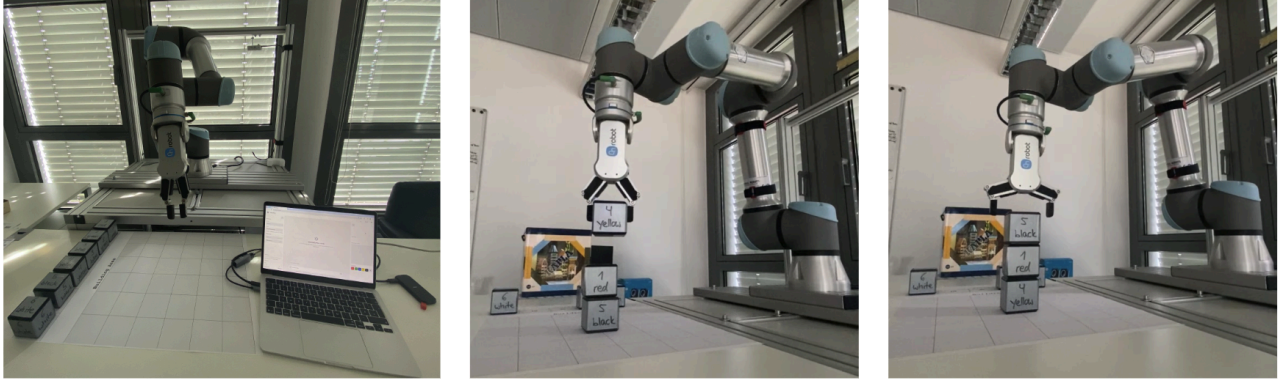


Figure 3. Three photographs of the physical cell: the whole setup seen from the operator's seat, a place in progress, and a finished three-cube tower.

The jaws open along one axis. At the reference orientation they open along one grid direction; rotated 90 degrees about the vertical they open along the other. A cube sitting one cell away on the open axis will be struck by a jaw. This single fact produces the free-axis rule (section 4.5), the most consequential piece of geometry in the system.

The arm homes to a known configuration on start-up, and the reference orientation is measured from the scene rather than hard-coded: the tool axis is straightened to vertical and the jaws snapped parallel to the world axes. Everything downstream is a translation of that one measured pose.

2.2 The workspace

The pitch is an internal detail. The agent addresses a plain 0 to 6 grid with integer stack levels and never sees a millimetre. Changing the physical plate is one configuration value, and it requires no change to the prompt, the tools, or anything else the model reads.

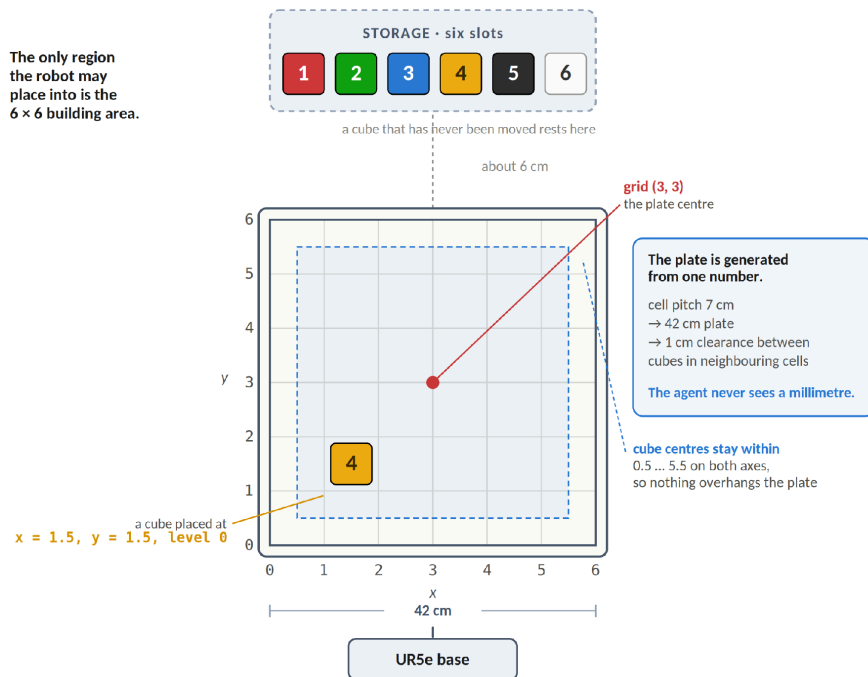


Figure 4. The storage row and the 6×6 building area, seen from above. The building area is the only region the robot may place into.

3. How the design arrived here

The current design is the third. Each stage was abandoned for a specific reason, and each left something behind that is still in the system.

3.1 Stage 1: one-shot plan generation, and why it breaks

The obvious first design. Describe the world and the available actions to the model, ask for a complete ordered plan, execute it. The model receives the cube catalog, the coordinate system, the geometry rules and the current layout, and returns the whole plan in one reply as a list of pick and place steps. The harness parses it and executes every step against the robot blind: results are recorded and shown in the UI, but no feedback ever returns to the model.

Four things go wrong, in rising order of seriousness.

- **It is an open loop.** The first action that fails desynchronises the plan from the world, and every later step then executes against a state the plan never anticipated.
- **Ordering constraints are invisible in a goal description.** This is the sharpest failure, and the figure below is the canonical case.
- **Intermediate-state simulation is hard.** When the world starts in a configuration the agent did not build, with a needed cube buried under two others or the goal cells occupied by the wrong cubes, a correct plan must simulate the whole trajectory: after I lift this, what is reachable, what did I just block, where does the thing I took off go? Every parking decision constrains later steps.

One-shot survives in the system as the cheapest baseline in every experiment. Without it in the matrix, an expensive agentic loop always wins by default.

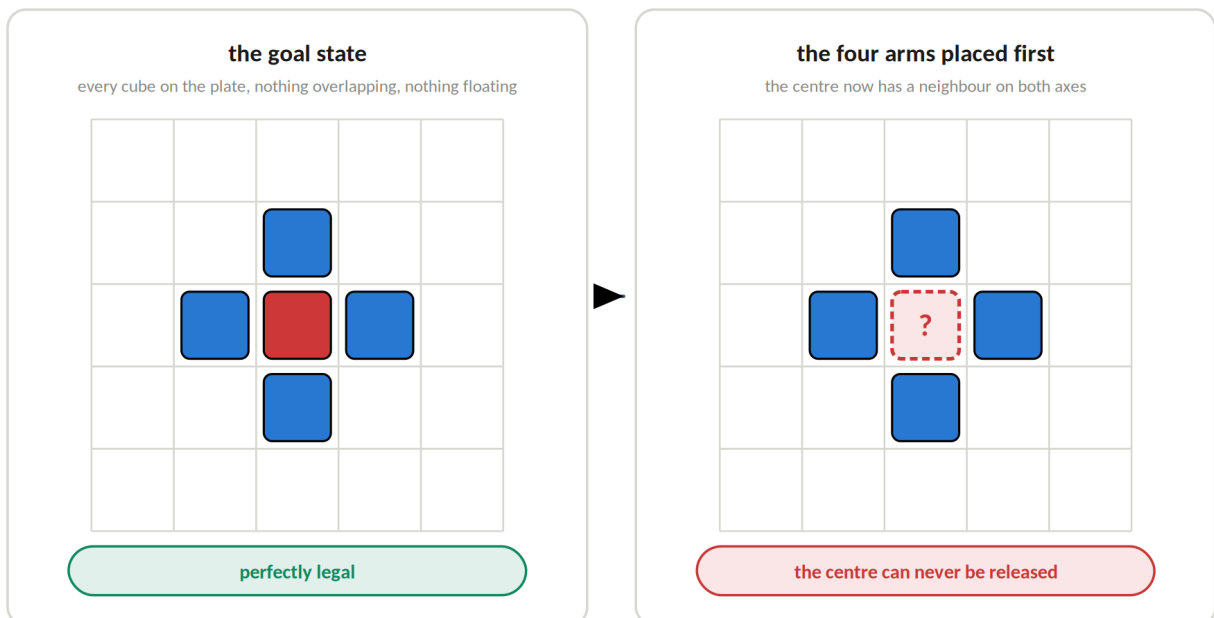


Figure 5. The ordering trap. The plus sign is a perfectly legal goal state, but if the four arms are placed first, the centre has a neighbour on both axes and the jaws can no longer open there. The goal is reachable; that order is not.

3.2 Stage 2: tools, without a world model

Instead of a plan, a typed action interface. The model calls pick and place one at a time, and the harness executes each call as it is issued.

This introduced the action space that is still in use (section 4.3), but it did not yet solve the real problem. The executor of the day had only a partial view of the world. It caught out-of-bounds placements after the arm had already started moving. It checked only one gripper axis, so a cell blocked on both axes silently drove a jaw into a cube. And it had no memory of what had been placed, so once a cube had been put down it could never be picked up again: the arm would return to the cube's empty storage slot and grasp air.

3.3 Stage 3: the agentic loop

The current design. Two changes, and they are inseparable.

1. The symbolic core owns the world and validates every proposed action before any motion (section 4).
2. The agent runs a feedback loop over the resulting acceptances and refusals (section 5).

Neither half works without the other. Validation without a loop just produces a stuck agent. A loop without validation just produces confident damage.

4. The symbolic core

4.1 The dividing line

The rule, stated as a test:

“If a question about the world has an exact answer computable from the world model, the language model is never asked it.”

Every item in the table below was, at some point in this project's history, either asked of the model or left unchecked. Each one moved into the symbolic layer after it caused a failure.

the question	answered by
What joint angles reach this pose?	the planner's inverse kinematics
Where is grid (3, 3) in metres?	the coordinate mapping
How high must the arm travel to clear the tallest stack?	the transit-height computation
Which way must the jaws face here?	the grasp-orientation selector
Where is every cube right now?	the tracked world model
Is this cell free? Is it supported? Is it on the plate?	the placement validator
Is this cube buried under another?	the world-model query
Did the agent actually achieve the goal?	the symbolic goal checker
What should be built, and what next?	the language model

4.2 The world model

The symbolic core keeps an authoritative record of the world: which cube is in the gripper, and the grid position and stack level of every cube resting in the building area. A cube in neither place is still at its storage slot.

Two things make this authoritative rather than merely a cache.

It is the only source that is correct. In this simulator, querying a cube's pose returns its original scene pose, because the simulator does not follow a cube as it is manipulated. So the simulator itself cannot answer “where is the red cube now?” after a placement. The tracked model can. The same property makes returning a cube to storage trivially correct: the scene pose is the storage slot, and therefore the release target.

Everyone reads it. The observation the model reasons over, the picture the UI draws, and the state the scorer grades are all views of this one record. There is exactly one world.

The observation merges each cube's colour and number in from the catalog, so the model reasons about “the red cube, number 1” rather than inferring meaning from an identifier string. The static part, the catalog, goes into the system prompt once. The volatile part, what is held and where things are, is delivered with the task and refreshed on request. It is deliberately kept out of the system prompt, where it would go stale after the first action and then sit at the top of the context contradicting everything below it.

4.3 The action space

Five tools. The coordinate ranges in their descriptions are generated from the live world, so they cannot drift from reality.

tool	arguments	what it does
get_state	none	returns the current world: what is in storage, what is placed and where, what is held. Moves nothing.
pick	cube id	grasps a cube from wherever it currently is, storage or a building-area cell
place	cube id, x, y, z	releases the held cube at a grid coordinate and stack level. x and y may be fractional; z is an integer level
store	cube id	returns a cube to its storage slot
done	reason	ends the task; the reason describes what was built and how it satisfies the request

The cube identifier is a fixed enumeration, so an invented cube name is a schema violation rather than a runtime error.

The schema is the contract, not the prompt. The system prompt is deliberately lean. It does not restate what the tool schema already says (argument ranges, the coordinate system), and it does not restate what feedback would teach anyway (pick before place, one cube at a time, stay in bounds). Duplicating those in prose spends context to create an opportunity for the two descriptions to disagree.

What the prompt does carry is the geometry that guides planning and that feedback would never suggest on its own: how support works, that side-by-side cubes need a full unit between centres, that a fractional coordinate lets an upper cube straddle two lower ones, and the free-axis rule. Those rules are shared word for word between the one-shot and the agentic prompt, so the two modes never describe different worlds.

One tool call per turn. Only the first call in a response is executed; any others come back with an explanation that they were skipped. This is a deliberate constraint, not a limitation of the executor. A model that emits five

calls at once has committed to all five without seeing the result of the first. That is one-shot planning wearing a tool-calling costume, with the added downside that it looks agentic in a transcript.

4.4 The validators

Every proposed action passes through a validator before any motion. Nothing in this stage touches the robot.

Picking a cube is refused when

check	what the agent is told
the cube id is unknown	the id is invalid, and the valid ones are listed
something is already held	“already holding cube_red; place it before picking another cube”
the cube is buried	“cube_black is buried: cube_white sits on top of it (level 1). Pick cube_white first, then cube_black.”
the jaws cannot open there	the free-axis refusal (section 4.5)

Placing a cube is refused when

check	what the agent is told
it is not the held cube	“holding cube_red, not cube_blue; only the held cube can be placed”
a coordinate is missing or not a number	which argument is missing
the target is off the plate	“(6, 3) is off the plate: a cube center must stay within 0.5 ... 5.5 on BOTH axes”
the level is not a whole number	what a stack level means, and what was received
the space is already built	“level 0 at (3, 3) is already occupied by cube_green. Choose a free cell, or stack on top of it at level 1.”
the space is not supported	“level 2 at (3, 3) has no cube directly below it (level 1 is empty), so the cube would float.”
the jaws cannot open there	the free-axis refusal

Two definitions are worth stating precisely, because the tasks depend on them.

- **Occupancy.** Two cubes on the same level collide when their centres are closer than one grid unit on both axes. Exactly adjacent is allowed.
- **Support.** The same overlap test, applied one level down. This is a footprint test, not an exact-column test, which is precisely what makes straddling work: a cube at $x = 2.5$ overlaps both the cube at $x = 2$ and the cube at $x = 3$, so either one supports it. The pyramid tasks depend on this.

Finally, any unexpected error from the planner or the motion layer is converted into feedback rather than a crash. The command server never dies on a bad command. A failure comes back as a readable message the agent can react to, and the session continues.

4.5 The free-axis rule

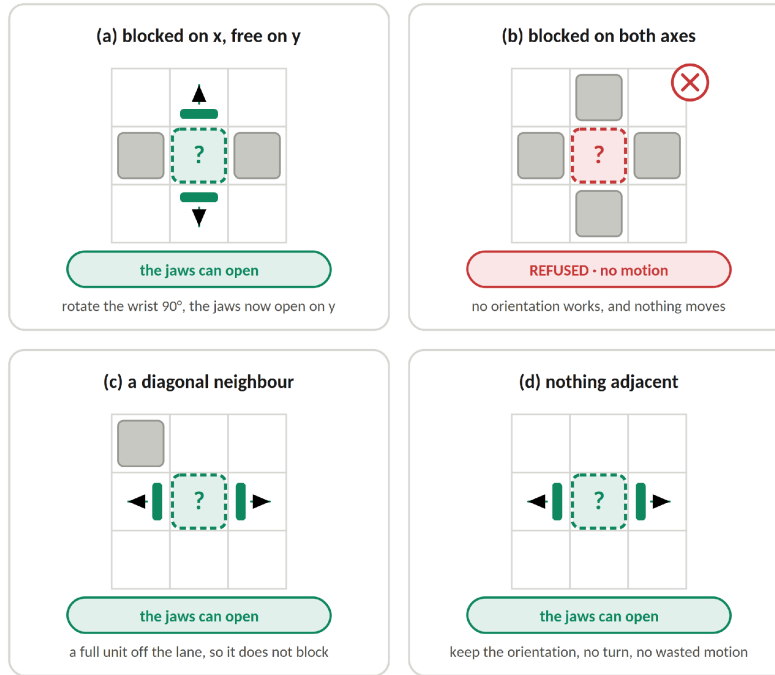


Figure 6. The four cases. Grey squares are cubes already standing, the dashed square is the cell the agent wants to grasp in or release into, and the green bars are the jaws.

The predicate asks whether a cube stands one cell away along a given axis, at the same level, in the same perpendicular lane. A cube further away is out of the jaws' path. A cube closer would be overlapping, which occupancy already forbids. And a purely diagonal neighbour is a full unit off in the perpendicular lane and does not block, which is case (c).

The selector then prefers the gripper's current orientation and turns only when that axis is blocked. So a run of grips whose current axis stays clear never re-turns, and a turn, there or back, is issued only when the target actually forces one.

Three things make this the signature example.

1. **It is invisible in the goal state.** A plus sign is a perfectly legal final configuration; only the order of construction can make its centre unreachable.
2. **It is exactly decidable.** Given the world model, “can the jaws open here?” is a two-line predicate with no uncertainty.
3. **Getting it wrong is a collision.** Before this check existed, only one axis was tested, and a cell blocked on both silently drove a jaw into a cube.

And when it refuses, it explains itself in a way that requires further reasoning rather than a mechanical retry.

4.6 Motion: coordinates, kinematics and collision-free transit

Three computations sit between a validated action and the robot.

Inverse kinematics. Never approached by the model. The symbolic layer computes a target pose for the arm's flange; the planner solves the joint angles and checks collisions.

The coordinate abstraction. The model addresses a 0 to 6 grid, and the symbolic layer maps that to world coordinates, applying the cell pitch and a rotation that accounts for the arm base being turned to the demo

orientation. The grid anchor is read from the scene, so moving the plate moves the coordinate system with it. There is no constant to update.

This abstraction is worth more than convenience. It makes the model's output dimensionless and bounded. “x must be within 0.5 ... 5.5” is checkable by comparison, whereas “0.4372 m” is not checkable at all without the full geometry. A placement error is off by a cell, not by an arbitrary float.

Collision-free transit. The planner does straight-line motion and has no path avoidance: it will not route around an obstacle. The naive consequence is brutal, because a diagonal hop from one target to the next cuts straight through any tower in between. The symbolic answer is a gantry path with a computed clearance height.

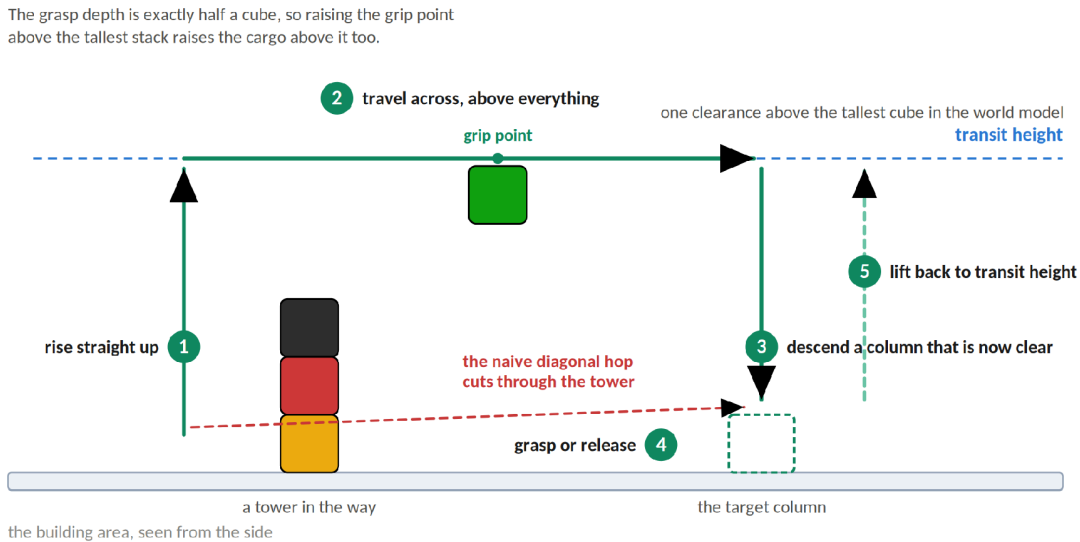


Figure 7. The gantry path. All horizontal travel happens above the tallest cube the world model knows about, so nothing between here and there can be clipped.

The travel height is the greater of one clearance above the tallest known cube and the approach height for this particular target. Because the grasp depth is exactly half a cube, a held cube's bottom face sits at the grip point, so raising the grip point above the tallest stack raises the cargo above it too. The only non-vertical segment of any grip runs at or above that height.

The 90 degree jaw turn is folded into the horizontal travel rather than performed on arrival. The grip point lies on the wrist axis and the tool stays vertical, so a pure turn is neutral in x and y. The move still ends exactly over the target and the rotation costs no extra time.

4.7 Rejection as a designed output

Every refusal the symbolic layer computes has two halves: a diagnosis, meaning what is wrong, and a remedy, meaning what to do about it. This is prompt engineering emitted by a validator, and it is the entire feedback channel the agent has to work with.

That split is also an experimental lever. The harness can deliver the full message, the diagnosis alone, or nothing but “action failed”, while leaving the world, the success flag and the loop's own behaviour byte-identical. Any difference between those arms is therefore attributable to the text alone.

4.8 What deliberately stays neural

The symbolic layer is not creeping toward a full symbolic planner. What only the model can do:

- **World knowledge.** That the German flag is black, red, gold, in that order. There is no table of flags in the system, and there should not be.
- **Interpreting an underspecified goal.** “A staircase a tiny toy figure could climb” has no formal specification. Neither does “a structure that defends the red block”.
- **Choosing what to do next.** Which cube to move, where to park a removed cube so it does not block a later step, whether to dismantle or work around.
- **Recovering from a refusal.** The remedy is a hint; turning it into a revised plan is inference.
- **Talking to the user.** Greeting, asking what to build, explaining what was built.

A compact way to put it: the symbolic layer defines the space of legal moves, and the model plays the game. A symbolic planner could search that space too, but it could not decide what “the German flag” is, and that is the half of the problem the language model is genuinely good at.

5. The agent harness

Everything that runs on the host: the model, the loop around it, and the question of how much that loop should be allowed to think.

5.1 The loop

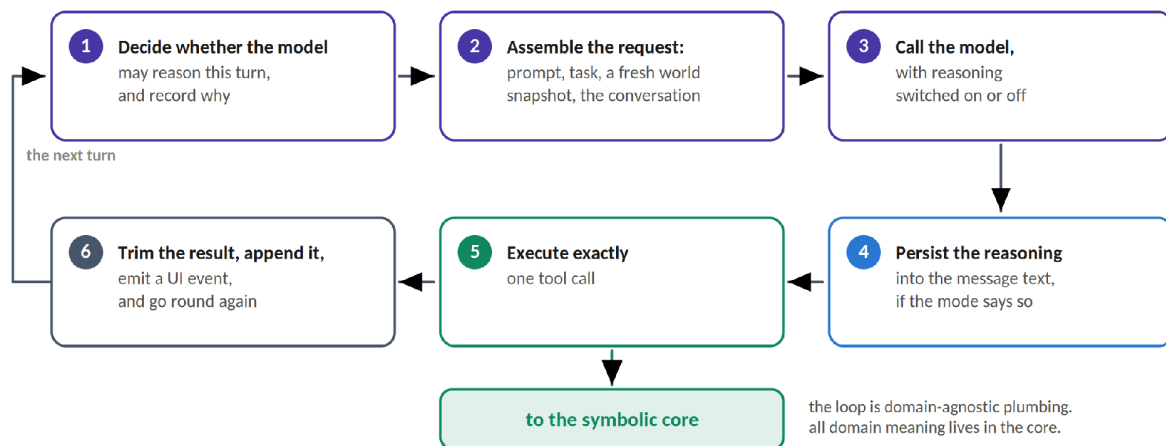


Figure 8. One turn of the agent loop. Note what step 5 does not include: any interpretation of the action.

The verdict in step 1 is announced to the UI before the call, so the reasoning that appears on screen can be labelled with the reason it was permitted: first turn, scheduled, or triggered by a failure.

The loop is domain-agnostic plumbing. All domain meaning lives in the symbolic core.

5.2 Two dimensions

Dimension 1: when may the model think? Reasoning is treated as a budgeted resource with a scheduling policy, not as an always-on property of the model. Three policies are in scope for this document.

policy	reasoning is enabled
never	not at all. The agent reads feedback and acts, but never deliberates
on error	on the first turn, and again whenever the world refuses an action
always	on every turn. Classic ReAct, and the upper bound

The error trigger fires once per failure: the loop consumes it, and a new refusal re-arms it. Otherwise it would latch on and never release.

Dimension 2: what survives to the next turn? This dimension exists because of a protocol detail that decides a surprising amount.

“A reasoning trace is not carried into the next request. The provider returns it, the log keeps it, and then it is gone. Anything the model worked out survives to the next turn only if it wrote it into its own message text.”

Some models do that natively. The 30B class routinely does not, as section 9.3 shows. So the harness supplies that memory externally, writing into the same channel.

cache	what is carried forward
none	nothing, beyond whatever the model itself wrote, which for Phoenix is empty
raw	the verbatim reasoning trace
summary	a distilled note of one to three lines, produced by one extra reasoning-off side-call

The summarizer is pinned to one model regardless of which model is driving, the same discipline as fixing the evaluation judge, so summarization quality stays constant across comparisons.

5.3 Session management

The chat is continuous. A follow-up like “now shift it a bit left” has context, and the world is not reset between tasks, so the structure stays as built.

But the context carried across tasks is compacted. While a task is running, the full turn-by-turn trail of reasoning, tool calls and feedback drives the loop. Once the task finishes, that trail collapses to a clean pair: the task text, and one assistant message summarising the outcome. The next task is re-seeded with a fresh world snapshot anyway, so none of the trail is needed to continue, and the full record is still preserved in the session log. The UI prints how much was compacted away after each task.

A task ends when the model calls done, at a step limit, or, when the model replies in prose instead of acting mid-build, which otherwise burns the whole step budget on nudges. A prose reply before the first action is a normal conversational turn: the agent may greet, answer a question, or ask what to build without entering the loop at all.

5.4 The four backends

The same harness drives four different worlds.

backend	where the world lives	the arm moves	needs
sim	in the container, driven by the simulator	simulated	the container stack
real	the same, driven by hardware	physically	the stack and robot preparation
fake	in the host process	no	nothing
replay	a recorded session	no	nothing, not even a model

The fake backend is faithful, not a mock. It reproduces the symbolic core's world model and all of its validation (bounds, occupancy, support, gripper reachability) with the same state transitions and the same feedback text. Only the motion is a no-op. This is possible precisely because the core already does all its validation before it moves anything.

The practical consequence is large: an entire evaluation campaign runs in seconds, with no containers and no arm. The honest caveat is that fake cannot reproduce a physical failure such as an unreachable pose, an unexpected collision or a dropped grasp. A green fake run means the orchestration is sound. It says nothing about the motion.

The replay backend plays a recorded session back through the live UI, with the same loop, the same live world model and the same events, and only the network call faked. It is how the system is shown with no key, no cost and no hardware.

6. The user interface

The browser demonstrator is the normal way to use this system. It is the place where the neuro-symbolic split becomes visible: you watch the model propose, and you watch the world answer.

6.1 The three-column layout

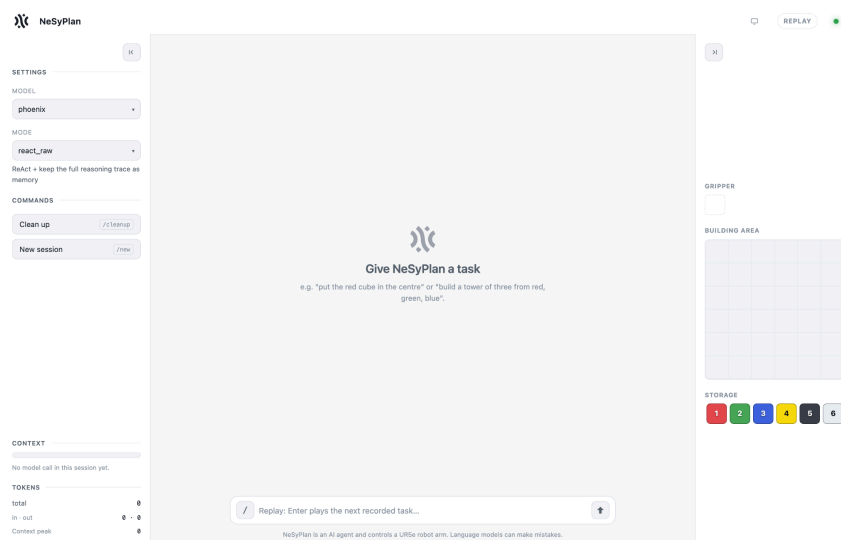


Figure 9. The interface before a task. Left: the controls. Middle: the conversation. Right: the world.

Left, the harness controls. The model (the endpoint follows the model, so the list spans providers) and the mode, with a one-line description below it. A clean-up button returns every cube to storage deterministically, without the model, for a reliable reset before a demo. Below that sit two live meters: context, split into what persists across tasks versus the current loop, and tokens, including the peak fraction of the model's context window used so far.

Middle, the conversation. The user's task, then every reasoning block, tool call and result, streamed as they happen.

Right, the world. The gripper, the 6×6 building area and the storage row. This is drawn from the same observation the model reads, so if the picture and the model disagree, the model is wrong, not the picture.

6.2 Rejection and recovery, on screen



Figure 10. The agent has boxed a cell in on both axes. The symbolic core refuses, and the arm does not move.

The model proposed `place(cube_id="cube_green", x=3.5, y=3, z=0)`. The core checked the world, found cubes standing one cell away on both the x-axis and the y-axis, and answered with the refusal.

Note also the three preceding turns. Each is a successful pick or place with its own confirmation, and each carries a “reasoning trace stored in content” note. The refusal is not an error state. It is one ordinary outcome of the loop, indistinguishable in mechanism from a success.

6.4 The world panel and the context inspector

The world panel is small, but it does real work. It shows the gripper with the held cube or an empty outline, the building area with cubes at their exact coordinates (stacks are drawn offset so the layers underneath stay visible), and the six storage slots, each of which empties when its cube is in play. Because it renders the same observation the model reads, it doubles as a debugging tool: a divergence between the panel and what the agent says it has built is immediate evidence that the model has lost track, and the symbolic layer has not.

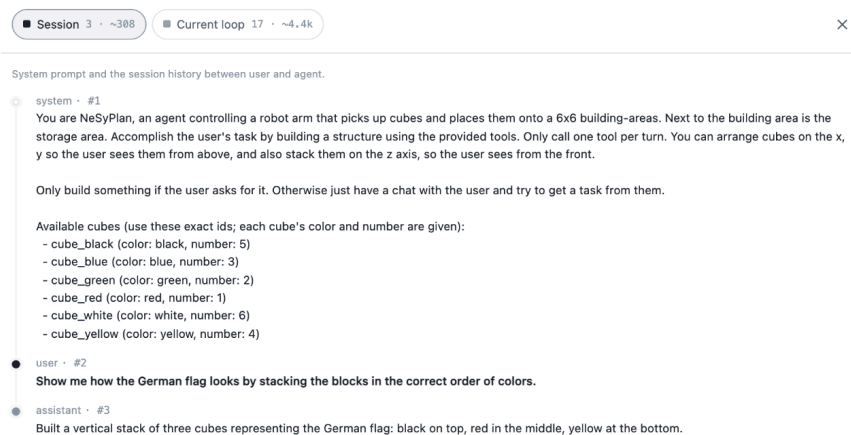


Figure 13. What the model actually sees, not a reconstruction.

Clicking either context meter opens the inspector, which shows the request as the model receives it, split into two tabs. The session tab holds the system prompt and the conversation that persists across tasks: the agent's identity, the instruction to call one tool per turn, the instruction not to build unless asked, and the cube catalog. The current loop tab holds the turn-by-turn trail of the task in progress.

This exists because the single most common source of confusion in an agentic system is the gap between what you think the model can see and what it actually receives. In `react_raw` and `react_summary`, the assistant messages here contain the persisted reasoning the cache wrote, so the inspector is also the direct way to check whether the memory mechanism is doing anything at all.

7. Why a language model must not steer a robot directly

7.1 Physical actions are not retryable

In software, a wrong action from a language model is usually cheap: catch the exception, retry, apologise. None of that holds here.

	a software tool call	a robot action
reversible?	usually	no. A toppled tower is not un-toppled
cost of a wrong number	an error	a collision at speed, into an expensive arm
detectable?	an exception	possibly nothing at all. The arm cheerfully grasps air
retry semantics	roughly idempotent	each retry is another physical motion

Add hallucination and specific failure shapes follow. A model asked for coordinates will produce coordinates: plausible ones, in the right format, for the wrong plate. A model asked to track six objects across twenty steps will lose track and will not say so. A model that has drifted will confidently continue.

7.2 The failure modes, and what neutralises each

Every row below is a failure this project actually encountered, and the symbolic mechanism that ended it.

failure	what it looks like physically	what neutralises it
hallucinated coordinates	the arm drives into the table or into free space	bounds and level validation, before motion
placing into an occupied cell	a cube dropped onto a cube; both fall	the occupancy check
placing onto nothing	a cube dropped from 6 cm; it bounces off the plate	the support check
grasping a buried cube	the jaws close on the cube above, or on the stack	the buried check
grasping or releasing in a boxed-in cell	a jaw driven into a neighbouring cube	the free-axis rule
diagonal travel through a tower	the held cube sweeps a standing stack over	the computed transit height
re-picking a placed cube	the arm returns to the empty storage slot and grasps air	the tracked world model
losing track of what is held	a placement issued with an empty gripper	the held record is authoritative
blind execution of a stale plan	steps 5 to 11 run against a world that stopped matching at step 4	the agentic loop: one action, then read the world
declaring success prematurely	the build is left half-finished and the user is told it is done	the symbolic checker disagrees, on the record
narrating instead of acting	the arm idles while tokens burn	the loop ends the task
five tool calls in one turn	five unvalidated motions from a single guess	only the first is executed

Read that table column-wise and the pattern is plain. Not one of these is fixed by a better prompt. Every fix is a predicate over a world model, evaluated before motion.

8. The evaluation harness

Every number in section 9 comes from four tasks and one scoring rule. Both are worth stating before the results, because the tasks are where the difficulty lives.

8.1 What a task is

A task is a prompt plus, wherever the goal is formally checkable, a symbolic checker. The checker is a predicate over the world model rather than a judgement about a picture, so it reads exact coordinates and returns a diagnostic sentence rather than a boolean. The loop does not grade itself. Task success is left empty while the episode runs and filled in afterwards by the scoring layer, from the same world state the agent was acting on.

8.2 The four rebuild tasks

Each of the four tasks starts from a layout the agent did not build, and each one breaks a different assumption.

R1 · excavate. The cube the goal needs sits at the bottom of a three-cube stack. Before the first action that contributes anything to the goal, the agent has to lift two cubes off it and put them down somewhere, and “somewhere” is the whole difficulty. A parking cell that the finished structure needs, or one that boxes in a cell

the agent will need later, turns a solvable task into a stuck one. Two removals and two parking decisions, all of them before the first goal step.

R2 · unbox. The goal cube starts with neighbours on both grasp axes. The obvious first action, picking it up, is refused before the arm moves, because the jaws have no free axis. The refusal names a remedy, clear one of those neighbours, and in this scenario that is deliberately not enough: freeing the cube takes a whole axis. A model that follows the sentence literally earns a second refusal and has to reason further.

R3 · repair. The goal is already built, in the wrong order. Every cell the finished structure needs is occupied by a cube that has to end up somewhere else, so nothing can be placed until something has been removed, and the order of the dismantling decides whether the rebuild is possible at all. It is the hardest of the four, and in the campaign of section 9 exactly one mode out of six solved it.

R4 · pyramid. The cubes are already on the plate, scattered across the build zone in the wrong places, and the target is a shape rather than a stack. It is the task that exercises straddling: an upper cube sits at a fractional coordinate over two lower ones.

What the four have in common is that the world contradicts the plan. None of them can be finished by placing cubes in a fresh space; every one of them makes the agent act on a state it did not produce and then answer for what that state makes impossible.

Other tiers exist for the demonstrator and for probing specific behaviours: knowledge and ordering (the flag), open construction (a pyramid, a staircase), precise arithmetic over the cube numbers, genuinely open-ended goals (“build a structure that defends the red block”), and the ordering traps of section 3.1, a plus sign and the letter T.

8.3 Scenarios: why tasks start mid-build

A scenario is a named starting layout the agent did not create. Nothing is hidden: both one-shot and agentic modes see the full layout. The difficulty is not observation. It is that a correct plan now requires simulating intermediate states, which is exactly where a one-pass plan breaks and per-action feedback starts paying for itself.

Every scenario is validated when it is loaded, against the same three invariants the executor enforces: nothing overlapping, everything supported, and every cube placeable in the given order, meaning its cell must still have a free grasp axis when its turn comes. A physically impossible starting layout is a typo, and it fails loudly rather than halfway through a campaign.

8.4 Metrics and campaign artifacts

Per episode the harness records tokens (total, prompt, completion, and summarizer overhead separately), the trace-length reasoning estimate, how many turns reasoned and by which trigger, refused actions, state queries, latency, and the final outcome.

A campaign writes a manifest (the full matrix, the code revision, the sampling settings, the judge model, so runs are self-describing), one transcript per cell including the verbatim reasoning, a results table, a success matrix and cost roll-up, a browsable report, and a rendering of each final state. Campaigns are resumable: re-running one skips finished cells.

9. Results

9.1 Which modes solve the tasks

mode	R1	R2	R3	R4	solved	tokens	reasoning	steps	refused
oneshot	✗	✗	✗	✓	1 / 4	5,298	3,878	7.5	2.8
on_error	✗	✓	✗	✓	2 / 4	66,434	13,958	23.5	7.0
act	✗	✗	✗	✗	0 / 4	70,255	1,291	30.0	16.2
react	✓	✓	✗	✓	3 / 4	102,222	43,380	24.8	4.8
react_summary	✗	✗	✗	✗	0 / 4	113,559	37,996	24.8	2.5
react_raw	✓	✓	✓	✓	4 / 4	122,674	10,306	15.8	2.0

Cheapest first. Reasoning is estimated from the length of the returned trace, because the endpoint reports no reasoning tokens, and steps counts actions taken before the task ended or the step limit was hit.

The floors are floors. The act mode never solved anything. It hit the 30-step limit on every task with 16.2 refused actions per run, an agent reading feedback and thrashing against it without deliberation. The oneshot mode managed one task out of four. Neither blind planning nor feedback without thinking survives a world that pushes back.

react_raw is the striking cell. It solved all four, and it did so with roughly 10.3k estimated reasoning tokens against react’s 43.4k, in 15.8 steps against 24.8, with 2.0 refused actions against 4.8, and in a third of the wall-clock time. It is the cheapest per unit of success and also the most accurate, which is the combination the next section explains.

Two other cells are worth holding on to, because they cut against the intuition. react_summary, which distils the trace instead of keeping it, spent the longest wall-clock time of any mode and carried 12.4k tokens of summarizer overhead, and solved nothing. And on_error, thinking only when the world objects, reached 2 out of 4 for two-thirds of react’s token cost, so deliberation is not linear in value.

9.2 Why react_raw wins

The mechanism is visible in the transcripts of the same campaign. Every assistant message across all six modes was inspected. Across 381 tool-calling turns, not one carried any message text.

mode	turns with a tool call	of those, with any message text	avg trace per turn
oneshot	0	n/a	11,499 chars
act	52	0	5,251
on_error	94	0	7,274
react	99	0	6,866
react_raw	62	0	2,662
react_summary	74	0	5,901

The control is in the first row. The oneshot mode makes no tool calls, because it is asked for a plan as text, and all eight of its turns wrote text. So this is not an inability to produce text. It is that when Phoenix calls a tool, it puts everything it worked out into the reasoning field and leaves the message empty. Combine that with the protocol fact from section 5.2, that the reasoning trace is not carried into the next request, and the consequence is structural.

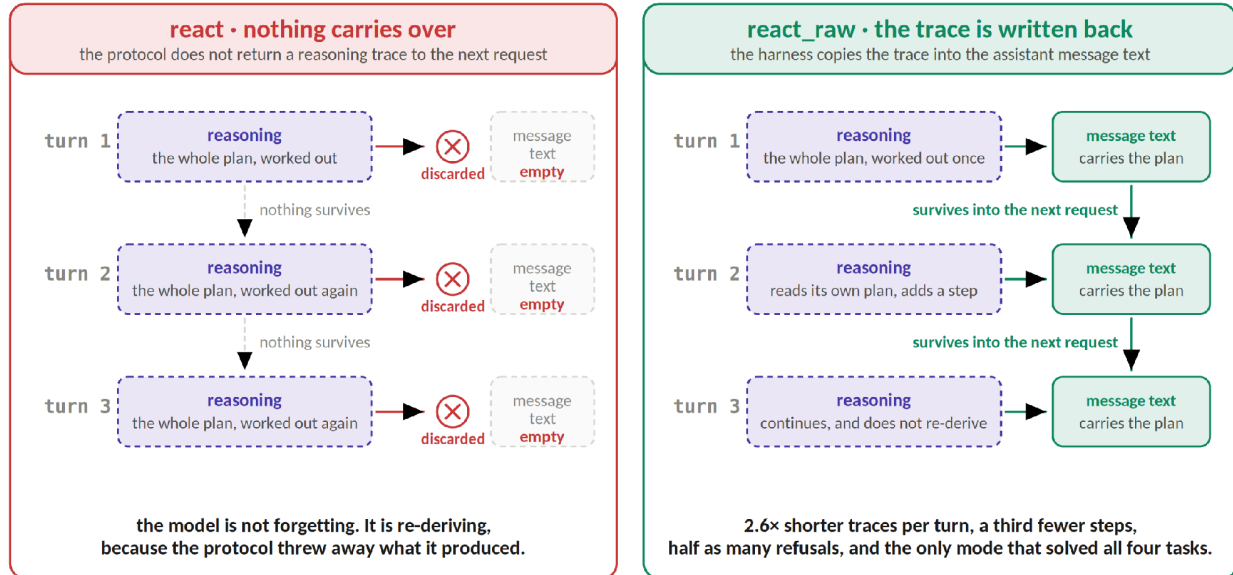


Figure 13. The mechanism. Without a memory channel the model re-derives its plan on every turn, because the artefact it produced last turn was thrown away by the protocol.

The model is not forgetting. It is re-deriving, every single turn. **react_raw** is the proof that this is the mechanism and not a story: it is the only mode that hands the trace back, and its average trace is 2,662 characters per turn against **react**'s 6,866, 2.6 times shorter, while needing fewer turns, 67 against 103. Given its own reasoning, the model stops re-deriving and starts continuing.

This also explains why more thinking was worse. **react** reasons on every turn with no memory, and it spent 4.2 times the estimated reasoning of **react_raw**, took 57 % more steps, was refused 2.4 times as often, and solved fewer tasks. Each independent re-derivation is a fresh sample from the model's plan distribution, consecutive samples disagree, and the agent then executes the disagreement: it places a cube, re-derives a plan that would have put it elsewhere, and picks it back up.

10. Limitations

The system is deliberately narrow, and three of its boundaries are worth naming plainly, because each one is a property of the design rather than a bug that is waiting to be fixed.

1. No perception. The world model is seeded once from the scene definition and is never corrected by sensing. Everything the agent is told about the world, and everything the validators check against, traces back to that file rather than to the table. This is what makes the validation exact, but it also means the system cannot tell the difference between a correct world and a confidently wrong one. If the physical cubes do not stand where the scene says they stand, every check still passes and every refusal is still well argued, only about a layout that no longer exists. The same gap swallows failures during a run: if a grasp slips, or a cube is nudged by hand between two turns, nothing notices, and the model carries on from a belief that has quietly stopped being true.

Confirming the physical layout before pointing a model at the arm is therefore not a formality, it is the only check that exists.

2. No path avoidance in the planner. The planner moves in straight lines and will not route around an obstacle, so the safety of every transfer rests on the gantry path: rise, travel above the tallest cube the world model knows about, descend into a column that is guaranteed clear. That construction makes collisions unrepresentable in this workspace, where the only obstacles are cubes the world model is already tracking and the plate is flat. It does not generalise. Put a fixture, a second robot or anything else the world model does not know about into the cell and the guarantee lapses, because the clearance height is computed from the model rather than observed from the scene.

3. No force or compliance control. Grasping is open-loop: the gripper is commanded to a width with a set force and reports nothing useful about what it actually holds. There is no feedback that would distinguish a clean grasp from a cube caught at an angle, from a jaw closing on air. Nothing in the system reacts to contact, so anything the arm meets on the way down is met at full commanded speed. For six identical light cubes on a flat plate this is an acceptable trade, and it is one of the reasons the task was kept that simple.